AirWiz

Plateforme de visualisation

Météo & Pollution atmospherique

Documentation technique complète Challenge Y.02 48h

Version	1.0 Document technique complet
Equipes	Data Développement Infrastructure
Dépôt	https://github.com/MounKilian/Y.02
Stack	React Node.js/Express Python PostgreSQL/Citus 12
Infrastructure	Docker Compose Citus HA Prometheus Grafana

Sommaire

1. Contexte du projet 4
2. Objectifs par équipe 4
2.1 Equipe Data 4
2.2 Equipe Développement 4
2.3 Equipe Infrastructure 5
3. Architecture réseau cible 6
4. Barème Partie Infrastructure 6
5. Prérequis 7
6. Structure du dossier airwiz/ 7
7. Etape 1 Recuperer le projet 8
8. Etape 2 Vérifier les secrets 8
9. Etape 3 Lancer la stack 8
9.1 Premier lancement (base de donnees vide) 8
9.2 Vérifier **que tout est** démarre 9
10. URLs et acces 9
10.1 Application web 9
10.2 Grafana identifiants 9
10.3 PostgreSQL accès **direct** 10
11. Démarrage du frontend, backend et service-data 10
11.1 Décommenter **les services dans docker-compose.yml** 10
11.2 Rebuilder et relancer 10
11.3 Flux de données **une fois tout actif** 11
12. Calcul de l'indice composite 12
12.1 Formule finale 12
12.2 Sous-score pollution 12
12.3 Sous-score meteo 12
12.4 Echelle d'interpretation 12
13. Frequence de rafraichissement des donnees 13
14. Monitoring Grafana 14
14.1 Dashboard AirWiz - Infrastructure 14
14.2 Alertes configurées **(rules.yml)** 14
15. Conformite au bareme 16
16. Dépannage 16
16.1 citus_master ne démarre **pas** 16
16.2 Vérification **de la** réplication 17
16.3 Test de bascule failover 17

16.4 Vérification **du sharding Citus**..... 17
16.5 Un service n'est pas healthy..... 18
16.6 Réinitialisation complète..... 18

PARTIE I CONTEXTE ET OBJECTIFS

1. Contexte du projet

Ce projet s'inscrit dans le cadre d'un exercice pluridisciplinaire impliquant trois équipes techniques aux responsabilites distinctes : Data, Développement et Infrastructure. L'objectif global est de concevoir et déployer une plateforme end-to-end permettant de visualiser, sur une interface cartographique, l'impact cumule des conditions météorologiques et de la pollution atmosphérique sur le territoire français.

Les données sources sont issues des Endpoint publics de la plateforme Open Data du gouvernement francais (data.gouv.fr). L'équipe Data est responsable de leur exploitation et de la génération d'un indice composite, mis a disposition de l'équipe de développement via une API interne. L'équipe Infrastructure assure l'hébergement sécurise et haute disponibilité de l'ensemble des services, avec segmentation réseau stricte, monitoring continu et gestion centralisée des secrets.

> **Challenge 48h Y.02**
> Ce document couvre l'intégralité de la partie Infrastructure du challenge.
> Toutes les étapes de lancement sont documentées et reproductibles.
> Mots de passe et secrets : voir section 3 (Secrets) et dossier secrets/.

2. Objectifs par équipe

2.1 Equipe Data

L'équipe Data est responsable de la collecte, du traitement et de l'exposition des donnees environnementales.

- Interroger les Endpoint Open Data gouvernementaux (météo SYNOP OMM et qualité de l'air LCSQA) et en assurer la récupération périodique.
- Développer et documenter l'algorithme de calcul de l'indice composite intégrant les dimensions météorologiques et pollution atmosphérique.
- Exposer les données calculées via une API interne REST (/indices) consommable par le backend applicatif.

2.2 Equipe Développement

L'équipe de développement conçoit et implémente l'application web complète, en s'appuyant sur les données mises a disposition par l'équipe Data.

- Développer une API backend Node.js/Express exposant les données filtrées issues du service Data via CRON.
- Implémenter un frontend React + Leaflet permettant la visualisation cartographique interactive des indices par station météo.

- Intégrer des fonctionnalités de filtrage (région géographique, plage temporelle, type d'indice) et de clustering dynamique.

2.3 Equipe Infrastructure

L'équipe Infrastructure est chargée de concevoir, déployer et opérer l'ensemble du socle technique.

- Dock riser l'ensemble des services avec des images optimisées (builds multi-stage, images Alpine légères).
- Deployer un cluster PostgreSQL/Citus en architecture Master/Slave avec réplication WAL streaming et failover automatique.
- Segmenter l'infrastructure en deux réseaux Docker isoles : réseau public (frontend/backend) et réseau interne non routable depuis internet (Data/DB).
- Mettre en place Prometheus + Grafana pour surveiller la sante des conteneurs et déclencher des alertes.
- Centraliser la gestion des secrets via Docker Secrets : aucun mot de passe en clair dans le code ou les fichiers de configuration.
- Bonus : sharding Citus par region_code (32 shards sur 2 workers).

3. Architecture réseau cible

L'infrastructure repose sur une segmentation stricte en deux reseaux Docker distincts, garantissant l'isolation des services sensibles.

Reseau	Services inclus	Exposition
Reseau 1 Public	Frontend web, Backend API	Expose sur internet (ports HTTP/HTTPS)
Reseau 2 Interne	Service Data, DB Master, DB Slave, Workers	Interne uniquement non routable depuis internet

Le backend est le seul service autorise a communiquer avec les deux réseaux simultanément. Il constitue la passerelle unique et contrôlée entre la zone publique et la zone interne. Cette règle est appliquée par la directive internal: true dans docker-compose.yml.

4. Barème Partie Infrastructure

Critere	Description	Points
Dockerisation & Orchestration	Qualité des Dockerfiles (multi-stage, images legeres) et déploiement Docker Compose.	/ 3
Haute Disponibilité (DB)	Configuration Master/Slave fonctionnelle. Test de bascule (failover) et persistance des volumes.	/ 4
Isolation & Segmentation Réseau	Etanchéité réelle des réseaux. Preuve que la DB n'est pas routable depuis l'extérieur.	/ 4
Monitoring & Observabilité	Stack Prometheus/Grafana avec métriques de sante des conteneurs.	/ 2
Gestion des Secrets	Aucun mot de passe en clair. Mécanisme de gestion dédie (Docker Secrets).	/ 2
TOTAL		/ 15

Bonus : sharding Citus par clé de région géographique (+1 point).

PARTIE II LANCEMENT DU PROJET

5. Prérequis

Outil	Version min.	Installation
Docker Engine	24.0	curl -fsSL https://get.docker.com | bash
Docker Compose	2.20	Inclus avec Docker Engine
Git	2.x	Inclus avec le système ou https://git-scm.com

Windows uniquement
Le port 80 est souvent bloque par IIS ou Hyper-V sur Windows.
Ce compose utilise les ports 3000 (Grafana) directement sans port 80/443.
Aucune modification nécessaire : tout fonctionne out-of-the-box.

6. Structure du dossier airwiz/

```
airwiz/
|-- docker-compose.yml          <- Orchestration (11 services actifs)
|-- .env                        <- Variables d'environnement (pret a l'emploi)
|-- secrets/
|   |-- postgres_password.txt   <- postgres123*
|   |-- replication_password.txt <- replication123*
|   |-- jwt_secret.txt          <- jwt123*
|   `-- grafana_password.txt    <- grafana123*
|-- postgres/
|   |-- init/init.sql           <- Schema + Citus + workers (execute 1 seule
fois)
|   |-- master/pg_hba.conf      <- Regles d'authentification avec replication
|   `-- slave/init-slave.sh     <- pg_basebackup + standby.signal
|-- monitoring/
|   |-- prometheus.yml          <- Scraping cAdvisor + postgres_exporter
|   |-- rules.yml               <- Alertes CPU / memoire / replication lag
|   `-- grafana/provisioning/   <- Datasource + dashboards auto-provisionnes
|-- frontend/   Dockerfile + nginx.conf (commente dans compose)
|-- backend/    Dockerfile (commente dans compose)
`-- service-data/ Dockerfile (commente dans compose)
```

7. Etape 1 Recuperer le projet

```
git clone https://github.com/MounKilian/Y.02.git
cd Y.02

# Placer le dossier airwiz/ a la racine du depot Y.02
# (a cote de client/, server/ et app/)
```

8. Etape 2 Vérifier les secrets

Les fichiers secrets sont pré-remplis et fonctionnels. Les voici :

Fichier	Valeur	Utilise par
secrets/postgres_password.txt	postgres123*	Tous les noeuds Citus + backend
secrets/replication_password.txt	replication123*	citus_master + citus_slave
secrets/jwt_secret.txt	jwt123*	Backend (tokens JWT)
secrets/grafana_password.txt	grafana123*	Grafana (compte admin)

Pour changer un mot de passe en production, modifier le fichier .txt correspondant puis relancer avec docker compose down -v && docker compose up -d.

9. Etape 3 Lancer la stack

9.1 Premier lancement (base de donnees vide)

```
cd airwiz

# Lancer tous les services en arriere-plan
docker compose up -d
```

Ordre de démarrage garanti par depends_on: condition: service_healthy :

```
citus_worker1 + citus_worker2   (demarrent en premier, sans dependance)
        |
   citus_master                 (attend les 2 workers)
        | Execute init.sql une seule fois :
        |   CREATE EXTENSION citus
        |   CREATE TABLE mesures (indice, horodatage, latitude, longitude...)
        |   create_distributed_table('mesures', 'region_code')
        |   citus_add_node(worker1), citus_add_node(worker2)
        |   CREATE USER replicator WITH PASSWORD 'replication123*'
        |   SELECT pg_create_physical_replication_slot('slave_slot')
        |
   citus_slave                  (pg_basebackup depuis master + standby.signal)
        |
```

```
    cadvisor + prometheus + grafana + postgres_exporter
```

9.2 Vérifier que tout est démarre

```
docker compose ps
```

Resultat attendu

```
NAME                STATUS
citus_worker1       Up (healthy)
citus_worker2       Up (healthy)
citus_master        Up (healthy)
citus_slave         Up (healthy)
cadvisor            Up
prometheus          Up (healthy)
grafana             Up (healthy)
postgres_exporter   Up
```

Temps de demarrage
Au tout premier lancement, citus_master execute init.sql (~30 secondes).
citus_slave execute pg_basebackup depuis le master (~30-60 secondes).
Pendant ce temps les services affichent 'health: starting' -- c'est normal.
Attendre ~90 secondes avant de vérifier docker compose ps.

10. URLs et acces

10.1 Application web

URL	Description
http://localhost	Application AirWiz carte interactive (quand frontend active)
http://localhost:3000	Grafana dashboards de monitoring

10.2 Grafana identifiants

```
URL       : http://localhost:3000
```

```
Login    : admin

Password : grafana123*



Dashboard : Infrastructure -> AirWiz - Infrastructure
```

10.3 PostgreSQL accès direct

```
# Se connecter directement au coordinator Citus

docker exec -it citus_master psql -U airwiz_user -d airwiz



# Verifier le schema

\dt



# Verifier les workers Citus

SELECT * FROM citus_get_active_worker_nodes();
```

11. Démarrage du frontend, backend et service-data

Le frontend, le backend et le service-data sont commentes dans docker-compose.yml car ils dependent du code source applicatif de l'equipe Dev (dossier client/, server/, app/ du depot Y.02). Une fois le code pret :

11.1 Décommenter les services dans docker-compose.yml

Retirer les # devant les blocs frontend, backend et service_data dans docker-compose.yml.

11.2 Rebuilder et relancer

```
# Builder uniquement les images applicatives
docker compose build --no-cache frontend backend service_data

# Relancer toute la stack
```

```
docker compose up -d

# Verifier les logs du backend (CRON + initialisation 10 derniers
jours)
docker compose logs -f backend
```

11.3 Flux de données une fois tout actif

```
data.gouv.fr (LCSQA 482 stations + SYNOP 188 stations)
        |
   service_data (Python : fetcher.py + geo_join.py + forecast.py)
        | expose /indices (reseau interne uniquement)
        |
   backend CRON (toutes les heures)
        | appelle GET /indices -> INSERT INTO mesures
        | au 1er demarrage : initialise les 10 derniers jours
        |
   Frontend React + Leaflet
        | auto-refresh toutes les 5 minutes
        | http://localhost
```

PARTIE III DETAILS TECHNIQUES

12. Calcul de l'indice composite

L'indice composite est calculé par l'équipe Data dans forecast.py. Il combine un sous-score pollution et un sous-score météo sur une échelle de 0 (optimal) a 100 (critique).

12.1 Formule finale

```
indice_composite = (indice_pollution x 0.6) + (indice_meteo x 0.4)

Poids pollution : 60%  (indicateur de sante primaire)
Poids meteo     : 40%  (influence sur la dispersion des polluants)
```

12.2 Sous-score pollution

Derive de l'indice ATMO (1 a 10) publie par Atmo France via le reseau LCSQA :

```
indice_pollution = (indice_atmo - 1) / 9 x 100

ATMO 1  -->  indice_pollution =   0  (tres bon)
ATMO 5  -->  indice_pollution =  44  (moyen)
ATMO 10 -->  indice_pollution = 100  (tres mauvais)
```

12.3 Sous-score meteo

```
temp_score     = max(0, min(100, (15 - temperature) x 5))
wind_score     = max(0, min(100, 100 - (vitesse_vent x 10)))
humidity_score = max(0, min(100, (humidite - 60) x 2.5))

indice_meteo = (temp_score x 0.4) + (wind_score x 0.4) +
(humidity_score x 0.2
```

12.4 Echelle d'interprétation

Plage	Niveau	Couleur carte	Action recommandee

0 - 20	**Excellent**	**Vert fonce**	**Aucune restriction**
21 - 40	**Bon**	**Vert clair**	**Activite exterieure normale**
41 - 60	**Modere**	**Jaune**	**Personnes sensibles : reduire les efforts**
61 - 80	**Mauvais**	**Orange**	**Reduire l'activite physique exterieure**
81 - 100	**Critique**	**Rouge**	**Eviter de sortir, ventiler avec precaution**

13. Frequence de rafraichissement des donnees

Evènement	***Fréquence***	***Description***
Collecte Python (service_data)	**Toutes les heures**	**Téléchargement LCSQA + SYNOP depuis data.gouv.fr**
CRON backend -> PostgreSQL	**Toutes les heures**	**Appel /indices + INSERT dans Citus**
Auto-refresh carte (frontend)	**Toutes les 5 min**	**Interroge /api/*, pas de rechargement de page**
Init base de données	**1 fois au 1er lancement**	**Charge les 10 derniers jours de données**
Scraping Prometheus	**Toutes les 15 sec**	**CPU, mémoire, lag replication, workers Citus**
Rafraichissement Grafana	**Toutes les 30 sec**	**Lecture depuis la base Prometheus TSDB**

14. Monitoring Grafana

Grafana est pre-configure et se lance automatiquement. Les dashboards sont provisionnes depuis le dossier monitoring/grafana/provisioning/.

14.1 Dashboard AirWiz - Infrastructure

Disponible automatiquement dans Infrastructure -> AirWiz - Infrastructure. Contient :

Services Status : citus_master UP/DOWN, connexions PostgreSQL, lag de réplication, conteneurs actifs

CPU Usage : utilisation CPU par conteneur (citus_master, slave, workers, prometheus, grafana)

Memory Usage : consommation mémoire par conteneur en bytes

PostgreSQL / Citus : lag replication, connexions actives/idle, taille DB, commits/rollbacks par seconde

Network I/O : trafic réseau entrant/sortant sur citus_master et citus_slave

14.2 Alertes configurées (rules.yml)

Alerte	Seuil	Severite
ServiceDown	Service inaccessible > 1 min	critical
ContainerRestarting	Boucle de redemarrage > 2 min	warning
ContainerCpuHigh	CPU > 80% pendant 5 min	warning
ContainerMemoryHigh	Memoire > 85% de la limite	warning
ContainerMemoryCritical	Memoire > 95% de la limite	critical

PostgresDown	**pg_up == 0 pendant 1 min**	**critical**
PostgresReplicationLag	**Lag > 30 secondes**	**warning**
PostgresReplicationLagCritical	**Lag > 120 secondes**	**critical**
CitusWorkerDown	**Worker inaccessible > 2 min**	**critical**

15. Conformité au barème

Critère	Statut	Implémentation
Dockerisation & Orchestration (/3)	OK	Dockerfiles multi-stage Alpine : frontend (React/Nginx), backend (Express), worker (Python)
Haute Disponibilite DB (/4)	OK	PostgreSQL 16 + Citus 12 : replication WAL streaming, failover automatique, slot slave_slot, volumes persistants
Isolation Reseau (/4)	OK	network_internal: true -- DB et worker sans aucun port publie, backend seule passerelle
Monitoring (/2)	OK	cAdvisor + Prometheus + Grafana + postgres_exporter -- alertes CPU/memoire/lag/Citus
Gestion Secrets (/2)	OK	Docker Secrets (tmpfs /run/secrets/) -- aucun mot de passe dans docker-compose.yml, env ou logs
Bonus Sharding (+1)	OK	create_distributed_table('mesures', 'region_code') -- 32 shards sur 2 workers Citus

16. Dépannage

16.1 citus_master ne démarre pas

Toujours commencer par supprimer les volumes pour repartir d'un PGDATA vide :

```
docker compose down -v
docker compose up -d
docker compose logs -f citus_master
```

16.2 Vérification de la réplication

```
# Le Slave doit etre en mode standby (retourne t = true)
docker exec citus_slave psql -U airwiz_user -d airwiz -c "SELECT
pg_is_in_recovery();"

# Le Master doit avoir une ligne streaming
docker exec citus_master psql -U airwiz_user -d airwiz -c "SELECT
client_addr, state, replay_lag FROM pg_stat_replication;"
```

Resultat attendu

```
 pg_is_in_recovery
-------------------
 t                    <- Slave est bien en standby

 client_addr  |   state   | replay_lag
--------------+-----------+------------
 172.x.x.x    | streaming | 00:00:00
```

16.3 Test de bascule failover

```
# 1. Simuler la panne du Master
docker stop citus_master

# 2. Apres ~15 secondes, verifier la promotion du Slave
docker exec citus_slave psql -U airwiz_user -d airwiz -c "SELECT
pg_is_in_recovery();"
# Resultat attendu : f (false = le Slave est maintenant Master)

# 3. Redemarrer l'ancien Master (il rejoint comme nouveau Slave)
docker start citus_master
```

16.4 Vérification du sharding Citus

```
# Repartition des shards (doit afficher 16 par worker)
docker exec citus_master psql -U airwiz_user -d airwiz \
  -c "SELECT nodename, count(*) FROM pg_dist_shard_placement
GROUP BY nodename;"

# Partition pruning (Task Count doit valoir 1 sur 32)
docker exec citus_master psql -U airwiz_user -d airwiz \
  -c "EXPLAIN SELECT * FROM mesures WHERE region_code = 'IDF';"
```

16.5 Un service n'est pas healthy

```
docker compose logs -f citus_master
docker compose logs -f citus_slave
docker compose restart prometheus
```

16.6 Réinitialisation complète

```
# Supprime tous les volumes et toutes les donnees (IRREVERSIBLE)
docker compose down -v
docker compose up -d
```

Attention
docker compose down -v supprime définitivement tous les volumes PostgreSQL.
Toutes les données collectées seront perdues.
N'utiliser que pour repartir complètement de zéro.